Intent assurance

Review the decisions and proof, not every generated line

Reader contract	What this guide provides
Audience	Technical owners and agent-workflow maintainers; basic delivery knowledge assumed.
Purpose	Detect misunderstood intent, contaminated inputs and unsupported delivery claims without exhaustive owner review.
Scope	Public industry accounts, general assurance principles and experimental guidance in this pack—not production certification or an audit of applications.

Bottom line: give agents executable obligations and independent checks; give the owner consequential exceptions and selected proof. Existing code, tests and documents inform the work but must not silently rewrite approved intent.

Evidence boundary: industry reports describe particular teams, not universal causal proof. General review questions are not observed defects; governance changes and synthetic exercises do not establish reduced owner effort or reliable future compliance.

Contents

1 What the evidence supports 2
1.1 Industry: assurance moves, it does not disappear 2
1.2 Traditional QA was already moving upstream 2
1.3 Failure patterns to check 2
2 The operating model 4
2.1 A small facade over inspectable evidence 4
2.2 Input integrity: structure supports authority 4
2.3 Code integrity: do not hide decisions in patches 4
2.4 Enforcement and change control 5
3 Materialization and validation 6
3.1 Existing owners, no parallel framework 6
3.2 Source authority is a content question 6
3.3 Evaluate the reviewer as well as delivery 6
3.4 What remains to prove 7
4 Sources 8

1 What the evidence supports

1.1 Industry: assurance moves, it does not disappear

The Claude Code team says, “We didn’t remove review. But we did move most of the reading to machines.” It reports Claude writing about 90% of that team’s own code, more than doubling merged PRs between June and September while the reverted share fell by over 40%; this is not a claim about all Anthropic coding, an incident-rate result or a controlled causal comparison. [1]

Layer	Reported mechanism
Before review	Runnable tests/builds/screenshots and actual outputs; fresh-context review; short goals/non-goals and code-owner failure/evidence criteria.
Agent review	Multiple finders; separate finding verification; bug/security findings resolved or justified; low-value style/cleanup noise filtered.
Enforcement	CI and open findings block merge; change-sensitive human requirements, trusted-author limits and sensitive-path exceptions; approvals tied to commits.
Recovery	Merge queues, internal flags, staged release and monitored revert/disable paths. Automatic circuit breakers tightening approval after deterioration were still a gap.
Reviewer quality	Daily evaluations from engineer-confirmed historical incidents, clean-case false-positive checks and comment outcomes; after three repair rounds, step back rather than repeat patches.

Humans retain direction, scope, taste, risky-code and architectural judgment, evidence interpretation and accountability. Internal low-risk merge arrangements do not change the public review product’s finding-only role, and are not a local approval policy to copy without authority. [1]

OpenAI describes one newly built internal-beta product with no manually written code, optional human PR review, agent-to-agent review and minimal blocking merge gates. It also describes substantial investment in isolated runnable applications, observability, machine-enforced architecture, a short entry map and linked knowledge, plus recurring cleanup; it does not quantify an escaped-defect comparison and explicitly leaves long-term coherence uncertain. [2]

These accounts differ in approval and flaky-test tolerance; neither establishes a universal safe merge recipe. OpenAI’s observation that agents replicate uneven existing patterns is particularly relevant to code as well as documentation contamination. [2]

1.2 Traditional QA was already moving upstream

DORA recommends peer review supported by continuous tests, integration and observability rather than remote approval boards reading every change. Its research found no evidence that more formal external review lowered change failure rates; that is not evidence that agents satisfy human segregation-of-duties or regulatory requirements. [3]

Continuous testing still includes human exploratory, usability and acceptance work. The recent addition is agents doing more building, observation, review and repair, with their own judgments needing calibration—not the disappearance of those assurance responsibilities. [4]

1.3 Failure patterns to check

These are general review questions, not findings about any particular implementation. Use them to choose a small amount of evidence against the actual authorized task.

Failure pattern	Small owner probe
Stale guidance becomes an accidental requirement	Which approved decision governs, and does the proposed behavior follow it rather than an obsolete description?
A successful command hides missing checks	Which critical claims were executed, failed, skipped or unavailable on this revision?
A component result is presented as complete delivery	What required end-to-end behavior remains unproved, and how is the handoff qualified?

A legitimate partial result need not be rejected. The failure is claiming unsupported scope or silently changing the agreed outcome; judging that requires the governing request and actual evidence, not an inference about the author's motives.

2 The operating model

2.1 A small facade over inspectable evidence

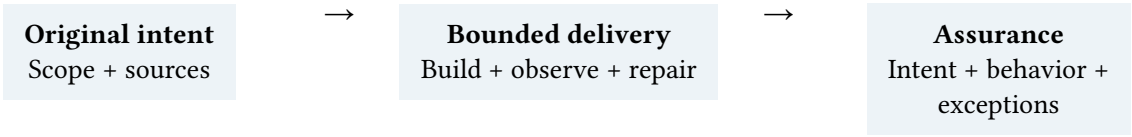


Figure 1: The owner inspects consequential exceptions and selected proof; detailed sources remain available. Unresolved material conflicts return before dependent implementation.

Before generating: retain original requests and compatible clarifications separately from mutable plans. State the outcome, must-not-change boundaries, relevant source/version and scope, plus one desired example and the closest plausible wrong example; this can be a few lines, not a new specification ladder.

During delivery: the builder owns implementation and genuine checks. The existing independent conformance checker compares the cumulative original requirements with the current result; responsibility for critical behavior proof is explicit, using a separate verifier where the contract or consequences require it.

At handoff: one card presents intent versus observed result; changed assumptions, removed/substituted/deferred behavior and its authority; one or two critical expected-versus-actual probes with evidence; and the precise unresolved decision with consequences and a recommendation. Detail is available on demand, not hidden or suppressed by the delivery lead.

The useful question is: "Show a case that distinguishes what I asked for from the nearest plausible misunderstanding—and demonstrate which behavior was delivered." A probe without an actual observation is a planned check, not proof; the owner can choose a different probe or inspect underlying evidence.

2.2 Input integrity: structure supports authority

Progressive disclosure means a small entry map leading to an owner guide and scoped detail/evidence. Drafts, history and transient outputs should remain available for investigation but off the default requirement route; folders and timestamps do not establish governing authority.

Maintain one normative claim with linked intentional summaries, rather than treating every duplicate sentence as waste. During affected work, reconcile source status, scope and supersession, report evidenced material conflicts as soon as found, and update relevant indexes/descriptions/outputs alongside the implementation.

Pause only work dependent on an unresolved conflict, not the whole repository. An already approved supersession is not a reason to ask again; a real retained-consumer or safety obligation is a reason to surface a contract defect rather than silently restore old behavior.

Mechanical checks can reveal missing targets and candidate duplicate text. They cannot prove that a source is authoritative, that two passages contradict each other or that an owner meant a particular outcome; semantic review still needs the original sources.

2.3 Code integrity: do not hide decisions in patches

Current code and passing tests are evidence of what exists, not automatic specifications of what should remain. They can encode legitimate undocumented domain, data, safety or external-consumer constraints; discover those rather than obeying or discarding them reflexively.

Trace intent ⟶ current behavior and consumers ⟶ mismatch ⟶ preserved/deleted or replaced semantics ⟶ applicable proof. Choose a local patch when it genuinely fits the authorized responsibility; refactor when the conflicting design must change within settled bounds; escalate unresolved consequential meaning or effects before embedding a workaround.

Ask: "What constraint does this workaround preserve, who requires it, and why is it still valid?" A compatibility adapter can be justified by a real retained interface; preserving an approved-retired path merely to keep tests green is not sufficient justification, and existing tests must not be weakened without approval.

2.4 Enforcement and change control

Executed, failed, skipped and unavailable checks must be distinct for each critical claim. Bind evidence and approvals to the actual source, artifact and configuration; changes invalidate affected proof, and a completed unit cannot stand in for a promised end-to-end outcome.

Repair in-scope defects without routine owner permission. Outcome, acceptance, scope or material-risk changes return through the applicable authority as explicit what/why/impact/options/recommendation proposals; QA success grants no merge, deployment, spending or acceptance rights.

Do not continue an unchanged review/repair loop indefinitely: diagnose by the third round at latest, earlier when a verifier breaks or a fingerprint repeats. Respect stricter existing budgets; this adds neither extra retries nor an arbitrary total-task cap or automatic human interruption when a safe revised path exists.

For authorized releases, observation signals, accountable recovery and a rollback/disable path matter as much as pre-release checks. The current pack's mandatory reviews and human-reserved rights remain the floor: no vendor-derived risk tiers, automatic merge path or newly scheduled service are installed by this guide.

3 Materialization and validation

3.1 Existing owners, no parallel framework

The implementation adds on-demand methods to existing bundles, with small authority/evidence/quality rule additions. The following paths are source-pack-relative locators, not links requiring a checkout to understand this detached PDF.

Skill bundle	Implemented responsibility
`working-principles`	`references/source-integrity.md`: affected intent/source/status/scope reconciliation and progressive disclosure.
`code-craft`	`references/code-intent-fit.md`: diagnose patch/refactor/escalation without an upfront style audit.
`team-operating-model`	`references/intent-assurance.md`: existing independent duties, compact exception card, critical proof, change/recovery/convergence and measures.
`reviewer`	`references/calibration.md` and synthetic CSV assets: known-defective/clean judgments with a separate grader, limits and exposure disclosure.
`project-foundation`	`references/context-integrity.md`: affected source/status/consumer reconciliation; accurate authoring-source identity.

The installation index and wiring map make these responsibilities discoverable; essential methods stay inside each skill bundle rather than requiring sibling identities. Native inspection, established repository checks where relevant and existing worker execution are sufficient; neither a custom link checker nor a new MCP is required for these methods.

3.2 Source authority is a content question

A reachable document can contain superseded instructions, and a current-looking file can conflict with approved intent. Verify the relevant content, governing decision, affected scope and actual consumer rather than substituting a link count for that judgment.

Native inspection or an established repository check can confirm a specific routing defect when one matters. That is supporting hygiene, not proof of semantic freshness, source authority or intent fidelity; create no custom tool without a demonstrated unmet need.

The useful evidence is the located conflict, its authority and status, the behavior it would change and the exact unresolved decision. Unavailable required sources remain unverified; a mechanical success must not hide that gap.

3.3 Evaluate the reviewer as well as delivery

Reviewer changes and any proposed reliance increase need representative known-defective and known-clean cases with independent grading. Keep original criteria and available capabilities fixed; hold expected judgments back from the reviewer, and disclose any exposure to gold labels, earlier answers or author narratives.

Compare consequential misses, false acceptance, invented harmful changes to clean cases, unnecessary blocking and actual effort. Reusing public synthetic cases is not secure holdout testing; confirmed private incidents need permission and suitable storage, not automatic capture into a shared corpus.

Synthetic cases can exercise the review method, but they do not establish field accuracy. If the prior method already handles a case correctly, that case cannot demonstrate an improvement; repeated exercises also have context/order confounders.

3.4 What remains to prove

Artifact checks can establish that current source routes resolve and this PDF is built from current sources. They cannot establish that every future agent will follow policy or that a real owner will spend less time repairing misunderstandings.

One-capability pilot should measure total owner review plus repair time, early intent-mismatch detection, post-handoff rework/escaped failures and reviewer noise. Select a distinguishing probe from that task's approved acceptance criteria, and record its expected result before execution; this guide supplies no application-specific test result.

Keep real mandatory gates and human decisions intact while gathering evidence. After one or two deliveries, retain, revise or remove additions based on actual benefit, false alarms and misses; do not equate report count, code volume, reviewer agreement or synthetic success with trust.

4 Sources

Public sources were inspected directly; multiple accounts from one company are not independent evidence lines. The source pack's approval, privacy and execution boundaries remain authoritative for its use; a vendor's merge policy is not permission here.

[1] A. Osmani, M. W. Lunt, and Q. Wang, "How the Claude Code team does code review." Accessed: Oct. 23, 2026. [Online]. Available: https://claude.dev/blog/solving-code-review-at-anthropic/

[2] R. Lopopolo, "Harness engineering: leveraging Codex in an agent-first world." Accessed: Oct. 23, 2026. [Online]. Available: https://openai.com/index/harness-engineering/

[3] DORA, "Streamlining change approval." Accessed: Oct. 23, 2026. [Online]. Available: https://dora.dev/capabilities/streamlining-change-approval/

[4] DORA, "Test automation." Accessed: Oct. 23, 2026. [Online]. Available: https://dora.dev/capabilities/test-automation/